

Application de Gestion de Scolarite

Documentation technique complète — API REST Spring Boot + JWT

1. Présentation

Application REST pour administrer un établissement scolaire : gestion des étudiants, enseignants, modules, inscriptions et notes avec authentification JWT.

Technologies : Java 17 | Spring Boot 3.2.5 | Spring Web | Spring Data JPA | Spring Security | MySQL | JWT (jjwt 0.11.5)

2. Architecture

Couche	Rôle	Classes
Entity	Modèle de données / tables BDD	Etudiant, Enseignant, Module, Inscription, Note
Repository	Accès base de données (JPA)	5 interfaces JpaRepository
Service	Logique métier	EtudiantService, EnseignantService, ModuleService, InscriptionService, NoteService
Controller	Endpoints REST	AuthController, EtudiantController, EnseignantController, ModuleController, InscriptionController, NoteController
Security	JWT Filter + UserDetailsService	JwtService, JwtAuthFilter, AdminUserDetailsService, SecurityConfig
DTO	Objets de transfert	LoginRequest, LoginResponse

3. Configuration MySQL

```
spring.datasource.url=jdbc:mysql://localhost:3306/scolarite_db
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
```

La base **scolarite_db** et toutes les tables sont créées automatiquement au démarrage.

4. Authentification JWT — Comment se connecter

L'application utilise **JSON Web Token (JWT)**. Il faut d'abord se connecter pour obtenir un token, puis utiliser ce token pour chaque requête protégée.

1 Se connecter — obtenir le token

Méthode	URL	Auth requise
POST	/api/auth/login	Non — public

Corps de la requête (JSON) :

```
{
  "email": "admin@scolarite.com",
  "password": "admin123"
}
```

Réponse en cas de succès (200 OK) :

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJyb2xlcyI6IltST0xFX0FETU0XSIsInN1YiI6ImFkbWluQHNjb2xhcmI0ZS5jb20iLCJpYXQiOiE3MTcwMDAwMDAsImV4cCI6MTcxNzA4NjQwMH0.XXX",
  "type": "Bearer",
  "email": "admin@scolarite.com",
  "role": "ROLE_ADMIN"
}
```

Mauvais email ou mot de passe : 401 Unauthorized — "Email ou mot de passe incorrect."

2 Utiliser le token dans Postman

Dans chaque requête protégée, ajouter le token dans l'en-tête HTTP :

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

Dans Postman :

Onglet	Champ	Valeur
Authorization	Type	Bearer Token
Authorization	Token	Coller le token reçu au login

Token valide : Accès autorisé — 200 OK ou 201 Created

Sans token ou token invalide : 403 Forbidden

3 Durée de validité

Le token est valide **24 heures**. Après expiration, refaire un login pour obtenir un nouveau token.

5. Règles de sécurité

Type de requête	Accès	Token requis
POST /api/auth/login	Public	Non
GET /api/**	Public — lecture libre	Non
POST /api/**	ADMIN uniquement	Oui
PUT /api/**	ADMIN uniquement	Oui
DELETE /api/**	ADMIN uniquement	Oui

6. Endpoints — Etudiants

Méthode	URL	Description	Auth
GET	/api/etudiants	Tous les étudiants	Non
GET	/api/etudiants/{id}	Etudiant par ID	Non
GET	/api/etudiants/matricule/{matricule}	Etudiant par matricule	Non
POST	/api/etudiants	Ajouter un étudiant	Oui
PUT	/api/etudiants/{id}	Modifier un étudiant	Oui
DELETE	/api/etudiants/{id}	Supprimer un étudiant	Oui

```
{
  "nom": "Benali", "prenom": "Yassine", "matricule": "ETU001",
  "email": "yassine@email.com", "dateNaissance": "2002-05-15",
  "filiere": "Informatique", "niveau": "L2"
}
```

7. Endpoints — Enseignants

Méthode	URL	Description	Auth
GET	/api/enseignants	Tous les enseignants	Non
GET	/api/enseignants/{id}	Enseignant par ID	Non
POST	/api/enseignants	Ajouter un enseignant	Oui
PUT	/api/enseignants/{id}	Modifier un enseignant	Oui
DELETE	/api/enseignants/{id}	Supprimer un enseignant	Oui

```
{
  "nom": "Mansouri", "prenom": "Karim",
  "email": "karim@ecole.com", "specialite": "Algorithmique"
}
```

8. Endpoints — Modules

Méthode	URL	Description	Auth
GET	/api/modules	Tous les modules	Non
GET	/api/modules/{id}	Module par ID	Non
GET	/api/modules/semestre/{s}	Modules d'un semestre	Non
POST	/api/modules	Ajouter un module	Oui
PUT	/api/modules/{id}	Modifier un module	Oui
PUT	/api/modules/{id}/enseignant/{eid}	Affecter un enseignant	Oui
DELETE	/api/modules/{id}	Supprimer un module	Oui

```
{
  "code": "INF301", "nom": "Algorithmique",
}
```

```
{
  "description": "Structures de données",
  "coefficient": 3,
  "semestre": 1
}
```

9. Endpoints — Inscriptions

Méthode	URL	Description	Auth
POST	/api/inscriptions/etudiant/{eid}/module/{mid}	Inscrire un étudiant	Oui
GET	/api/inscriptions/etudiant/{eid}	Modules d'un étudiant	Non
GET	/api/inscriptions/module/{mid}	Etudiants d'un module	Non
DELETE	/api/inscriptions/{id}	Supprimer une inscription	Oui

Pas de body requis — les IDs sont dans l'URL.

10. Endpoints — Notes

Méthode	URL	Description	Auth
POST	/api/notes	Ajouter une note	Oui
PUT	/api/notes/{id}	Modifier une note	Oui
DELETE	/api/notes/{id}	Supprimer une note	Oui
GET	/api/notes/etudiant/{id}	Toutes les notes d'un étudiant	Non
GET	/api/notes/etudiant/{id}/semestre/{s}	Notes par semestre	Non
GET	/api/notes/etudiant/{id}/moyenne	Moyenne générale	Non
GET	/api/notes/etudiant/{id}/moyenne/semestre/{s}	Moyenne par semestre	Non
GET	/api/notes/etudiant/{id}/resultats	Résultats + mention	Non

```
// Ajouter une note
{ "etudiantId": 1, "moduleId": 1, "note": 15.5, "annee": "2024-2025" }

// Modifier une note
{ "note": 17.0 }
```

11. Système de mentions

Moyenne	Mention
>= 16 / 20	Très Bien
>= 14 / 20	Bien
>= 12 / 20	Assez Bien
>= 10 / 20	Passable
< 10 / 20	Insuffisant

12. Gestion des erreurs

Situation	Code HTTP
Ressource créée avec succès	201 Created
Requête GET réussie	200 OK
Suppression réussie	204 No Content
Etudiant / Enseignant / Module / Note introuvable	404 Not Found
Etudiant déjà inscrit au même module	400 Bad Request
Email ou mot de passe incorrect	401 Unauthorized
Token absent ou invalide sur endpoint protégé	403 Forbidden

13. Lancer l'application

```
# Etape 1 : créer la base MySQL si elle n'existe pas
mysql -u root -proot -e "CREATE DATABASE IF NOT EXISTS scolartite_db;"

# Etape 2 : libérer le port 8080 si nécessaire
fuser -k 8080/tcp

# Etape 3 : lancer l'application
./mvnw spring-boot:run
```

